

# Botzer\_adv3\_panorama-stitching-solution

August 27, 2025

```
[1]: from __future__ import division, print_function
      %matplotlib inline
```

```
[2]: # Imports for functions

      from PIL import Image

      import skimage as ski
      import numpy as np
      import matplotlib.pyplot as plt
      from matplotlib import patches
      import cv2 as cv

      # Showing the results of scipy computations
      import pandas as pd

      cv.__version__, ski.__version__
```

```
[2]: ('4.8.1', '0.22.0')
```

## 1 scikit-image advanced panorama tutorial

Enhanced from the original demo as featured in [the scikit-image paper](#).

Multiple overlapping images of the same scene, combined into a single image, can yield amazing results. This tutorial will illustrate how to accomplish panorama stitching using scikit-image, from loading the images to cleverly stitching them together.

### 1.0.1 First things first

Import NumPy and matplotlib, then define a utility function to compare multiple images

```
[3]: import numpy as np
      import matplotlib.pyplot as plt

      def compare(*images, **kwargs):
          """
```

*Utility function to display images side by side.*

*Parameters*

-----

*image0, image1, image2, ... : ndarray*

*Images to display.*

*labels : list*

*Labels for the different images.*

"""

```
f, axes = plt.subplots(1, len(images), **kwargs)
```

```
axes = np.array(axes, ndmin=1)
```

```
labels = kwargs.pop('labels', None)
```

```
if labels is None:
```

```
    labels = [''] * len(images)
```

```
for n, (image, label) in enumerate(zip(images, labels)):
```

```
    axes[n].imshow(image, interpolation='nearest', cmap='gray')
```

```
    axes[n].set_title(label)
```

```
    axes[n].axis('off')
```

```
f.tight_layout()
```

## 1.0.2 Load data

The ImageCollection class provides an easy and efficient way to load and represent multiple images. Images in the ImageCollection are not only read from disk when accessed.

Load a series of images into an ImageCollection with a wildcard, as they share similar names.

```
[4]: # Read in the images
pano0 = ski.io.imread('Library1.jpg')
pano1 = ski.io.imread('Library2.jpg')
pano2 = ski.io.imread('Library3.jpg')

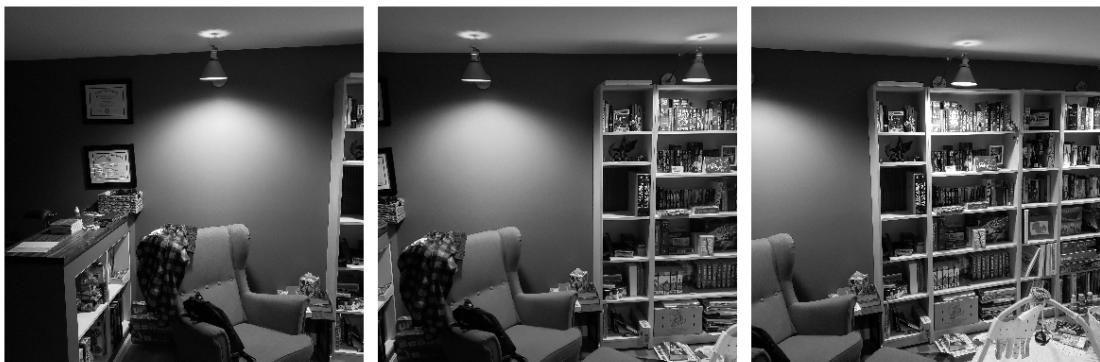
# Convert these to grayscale knowing the SIFT needs grayscale
pano0 = ski.color.rgb2gray(pano0)
pano1 = ski.color.rgb2gray(pano1)
pano2 = ski.color.rgb2gray(pano2)

# For some reason these get rotated from my phone
pano0 = ski.transform.rotate(pano0, -90)
pano1 = ski.transform.rotate(pano1, -90)
pano2 = ski.transform.rotate(pano2, -90)
```

```
[5]: pano_imgs = [pano0, pano1, pano2]
```

Inspect these images using the convenience function `compare()` defined earlier

```
[6]: compare(*pano_imgs, figsize=(12, 10))
```



Credit: Images of Private Arch and the trail to Delicate Arch in Arches National Park, USA, taken by Joshua D. Warner. License: CC-BY 4.0

---

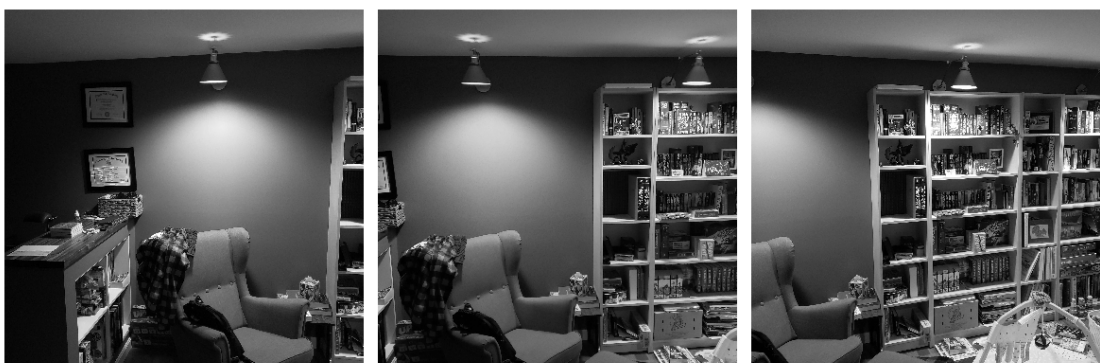
## 2 0. Pre-processing

This stage usually involves one or more of the following:

- \* Resizing, often downscaling with fixed aspect ratio
- \* Conversion to grayscale, as many feature descriptors are not defined for color images
- \* Cropping to region(s) of interest

For convenience our example data is already resized smaller, and we won't bother cropping. However, they are presently in color so conversion to grayscale with `skimage.color.rgb2gray` is appropriate.

```
[7]: # View the results  
compare(pano0, pano1, pano2, figsize=(12, 10))
```



### 3 1. Feature detection and matching

We need to estimate a projective transformation that relates these images together. The steps will be

1. Define one image as a *target* or *destination* image, which will remain anchored while the others are warped
2. Detect features in all three images
3. Match features from left and right images against the features in the center, anchored image.

In this three-shot series, the middle image `pano1` is the logical anchor point.

We detect “Oriented FAST and rotated BRIEF” (ORB) features in both images.

**Note:** For efficiency, in this tutorial we’re finding 800 keypoints. The results are good but small variations are expected. If you need a more robust estimate in practice, run multiple times and pick the best result *or* generate additional keypoints.

```
[8]: from skimage.feature import SIFT

# Initialize ORB
# 800 keypoints is large enough for robust results,
# but low enough to run within a few seconds.
sift = SIFT(n_octaves=3, n_scales=2)

# Detect keypoints in pano0
sift.detect_and_extract(pano0)
keypoints0 = sift.keypoints
descriptors0 = sift.descriptors

# Detect keypoints in pano1
sift.detect_and_extract(pano1)
keypoints1 = sift.keypoints
descriptors1 = sift.descriptors

# Detect keypoints in pano2
sift.detect_and_extract(pano2)
keypoints2 = sift.keypoints
descriptors2 = sift.descriptors
```

Match features from images 0 <-> 1 and 1 <-> 2.

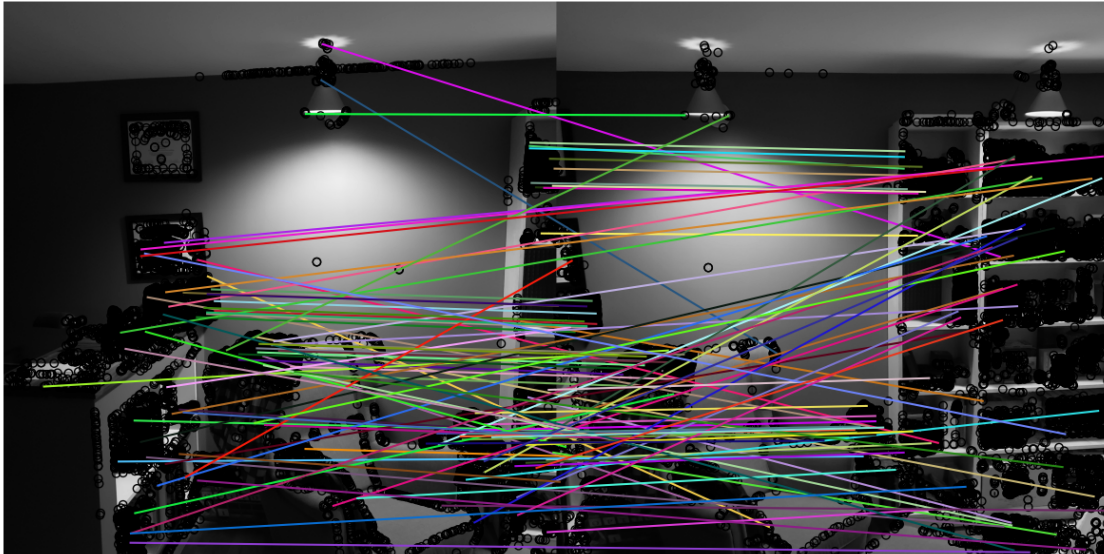
```
[9]: from skimage.feature import match_descriptors

# Match descriptors between left/right images and the center
matches01 = match_descriptors(descriptors0, descriptors1, cross_check=True)
matches12 = match_descriptors(descriptors1, descriptors2, cross_check=True)
```

Inspect these matched features side-by-side using the convenience function `skimage.feature.plot_matches`.

```
[10]: from skimage.feature import plot_matches
fig, ax = plt.subplots(1, 1, figsize=(15, 12))

# Best match subset for pano0 -> pano1
plot_matches(ax, pano0, pano1, keypoints0, keypoints1, matches01[::30])
ax.axis('off');
```



Most of these line up similarly, but it isn't perfect. There are a number of obvious outliers or false matches.

```
[11]: fig, ax = plt.subplots(1, 1, figsize=(15, 12))

# Best match subset for pano2 -> pano1
plot_matches(ax, pano1, pano2, keypoints1, keypoints2, matches12[::30])
ax.axis('off');
```



Similar to above, decent signal but numerous false matches.

## 4 2. Transform estimation

To filter out the false matches, we apply RANdom SAMple Consensus (RANSAC), a powerful method of rejecting outliers available in `skimage.transform.ransac`. The transformation is estimated iteratively, based on randomly chosen subsets, finally selecting the model which corresponds best with the majority of matches.

We need to do this twice, once each for the transforms left -> center and right -> center.

```
[12]: from skimage.transform import ProjectiveTransform
      from skimage.measure import ransac

      # Select keypoints from
      # * source (image to be registered): pano0
      # * target (reference image): pano1, our middle frame registration target
      src = keypoints0[matches0[:, 0]][:, :-1]
      dst = keypoints1[matches0[:, 1]][:, :-1]

      model_robust01, inliers01 = ransac((src, dst), ProjectiveTransform,
                                         min_samples=4, residual_threshold=1,
                                         ↪max_trials=300)

      # Select keypoints from
      # * source (image to be registered): pano2
      # * target (reference image): pano1, our middle frame registration target
```



```
src = keypoints2[matches12[:, 1]][:, ::-1]
dst = keypoints1[matches12[:, 0]][:, ::-1]

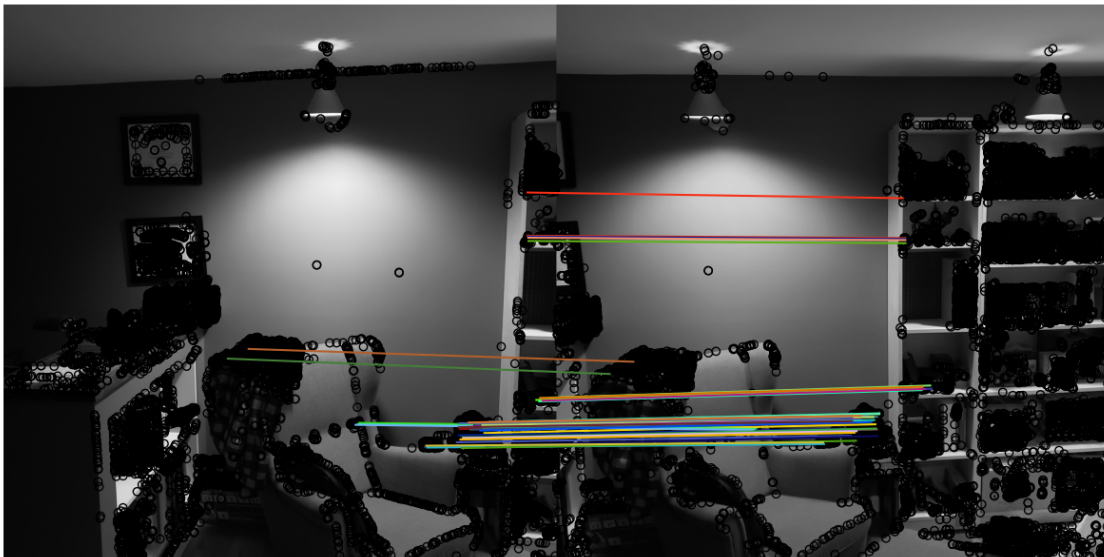
model_robust12, inliers12 = ransac((src, dst), ProjectiveTransform,
                                   min_samples=4, residual_threshold=1,
                                   ↪max_trials=300)
```

The `inliers` returned from RANSAC select the best subset of matches. How do they look?

```
[13]: fig, ax = plt.subplots(1, 1, figsize=(15, 12))

# Best match subset for pano0 -> pano1
plot_matches(ax, pano0, pano1, keypoints0, keypoints1, matches01[inliers01])

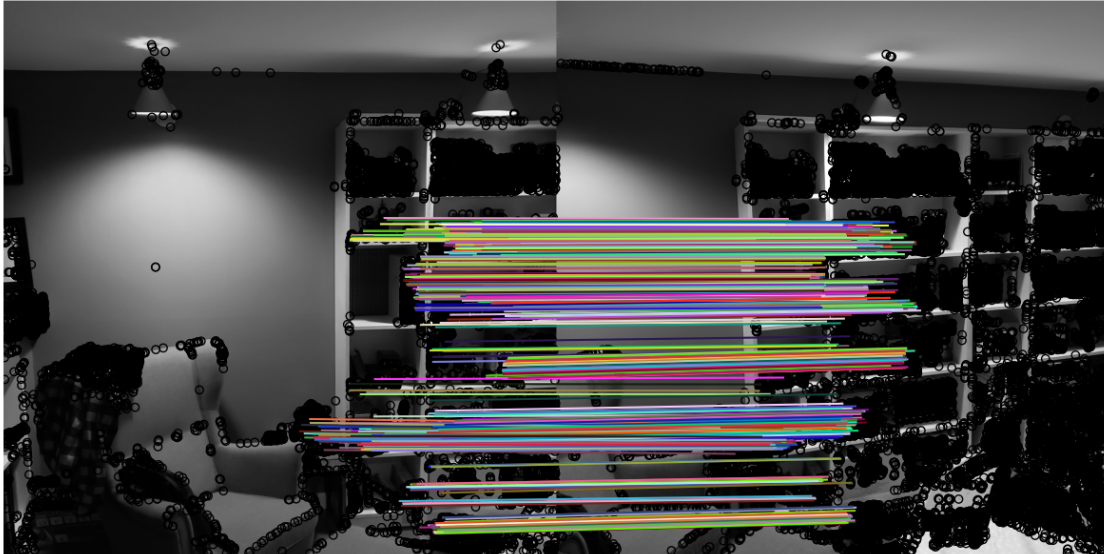
ax.axis('off');
```



```
[14]: fig, ax = plt.subplots(1, 1, figsize=(15, 12))

# Best match subset for pano2 -> pano1
plot_matches(ax, pano1, pano2, keypoints1, keypoints2, matches12[inliers12])

ax.axis('off');
```



Most of the false matches are rejected!

---

## 5 3. Warping

Next, we produce the panorama itself. We must *warp*, or transform, two of the three images so they will properly align with the stationary image.

### 5.0.1 Extent of output image

The first step is to find the shape of the output image to contain all three transformed images. To do this we consider the extents of all warped images.

```
[15]: from skimage.transform import SimilarityTransform

# Shape of middle image, our registration target
r, c = pano1.shape[:2]

# Note that transformations take coordinates in (x, y) format,
# not (row, column), in order to be consistent with most literature
corners = np.array([[0, 0],
                    [0, r],
                    [c, 0],
                    [c, r]])

# Warp the image corners to their new positions
warped_corners01 = model_robust01(corners)
warped_corners12 = model_robust12(corners)
```



```

# Find the extents of both the reference image and the warped
# target image
all_corners = np.vstack((warped_corners01, warped_corners12, corners))

# The overall output shape will be max - min
corner_min = np.min(all_corners, axis=0)
corner_max = np.max(all_corners, axis=0)
output_shape = (corner_max - corner_min)

# Ensure integer shape with np.ceil and dtype conversion
output_shape = np.ceil(output_shape[:-1]).astype(int)

```

## 5.0.2 Apply estimated transforms

Warp the images with `skimage.transform.warp` according to the estimated models. A shift, or *translation* is needed to place as our middle image in the middle - it isn't truly stationary.

Values outside the input images are initially set to -1 to distinguish the “background”, which is identified for later use.

**Note:** `warp` takes the *inverse* mapping as an input.

```

[16]: from skimage.transform import warp

# This in-plane offset is the only necessary transformation for the middle image
offset1 = SimilarityTransform(translation= -corner_min)

# Translate pano1 into place
pano1_warped = warp(pano1, offset1.inverse, order=3,
                    output_shape=output_shape, cval=-1)

# Acquire the image mask for later use
pano1_mask = (pano1_warped != -1) # Mask == 1 inside image
pano1_warped[~pano1_mask] = 0      # Return background values to 0

```

Warp left panel into place

```

[17]: # Warp pano0 (left) to pano1
transform01 = (model_robust01 + offset1).inverse
pano0_warped = warp(pano0, transform01, order=3,
                    output_shape=output_shape, cval=-1)

pano0_mask = (pano0_warped != -1) # Mask == 1 inside image
pano0_warped[~pano0_mask] = 0      # Return background values to 0

```

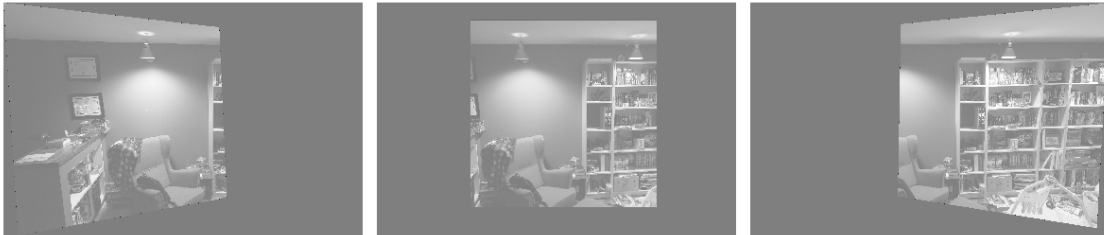
Warp right panel into place

```
[18]: # Warp pano2 (right) to pano1
transform12 = (model_robust12 + offset1).inverse
pano2_warped = warp(pano2, transform12, order=3,
                    output_shape=output_shape, cval=-1)

pano2_mask = (pano2_warped != -1) # Mask == 1 inside image
pano2_warped[~pano2_mask] = 0     # Return background values to 0
```

Inspect the warped images:

```
[19]: compare(pano0_warped, pano1_warped, pano2_warped, figsize=(12, 10));
```

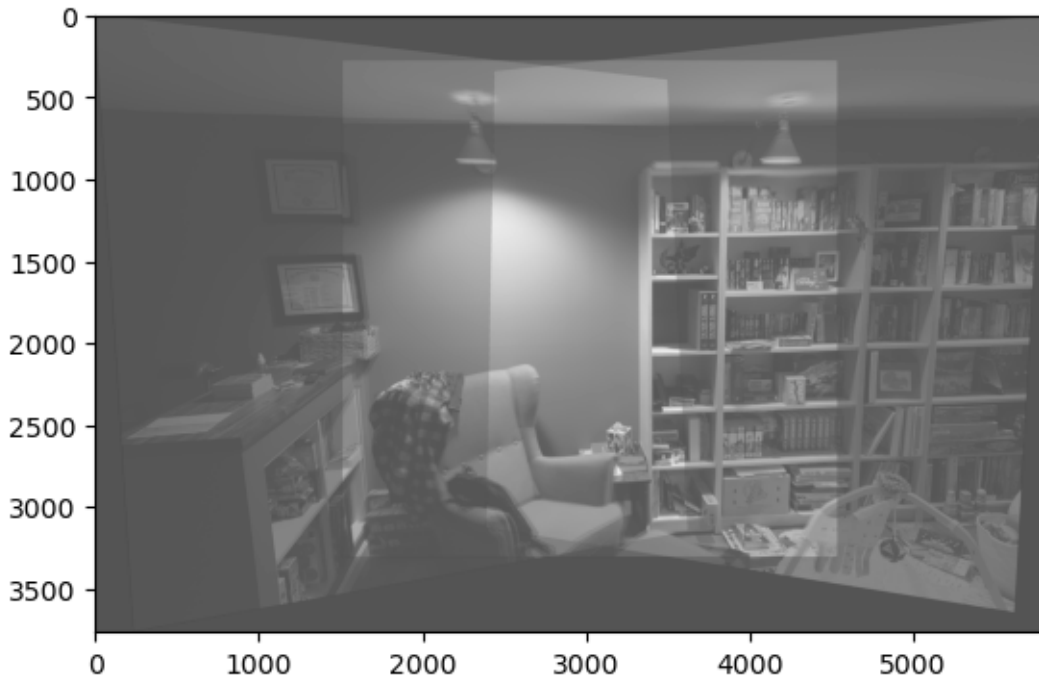


## 6 4. Combining images the easy (and bad) way

This method simply

1. sums the warped images
2. tracks how many images overlapped to create each point
3. normalizes the result.

```
[22]: # Add the three images together. This could create dtype overflows!
# We know they are are floating point images after warping, so it's OK.
merged = (pano0_warped + pano1_warped + pano2_warped)
plt.imshow(merged, cmap='gray');
```



```
[28]: np.min(pano0_warped)
```

```
[28]: -0.9999973264860373
```

```
[29]: # Track the overlap by adding the masks together
      overlap = (pano0_mask * 1.0 + # Multiply by 1.0 for bool -> float conversion
                  pano1_mask +
                  pano2_mask)
```

```
[33]: overlap.shape
```

```
[33]: (3763, 5824)
```

```
[34]: np.max(overlap)
```

```
[34]: 3.0
```

```
[31]: # Normalize through division by `overlap` - but ensure the minimum is 1
      normalized = merged / np.maximum(overlap, 1)
```

Finally, view the results!

```
[32]: fig, ax = plt.subplots(figsize=(12, 12))

      ax.imshow(normalized, cmap='gray')
```

```
plt.tight_layout()
```



---

**What happened?!** Why are there nasty dark lines at boundaries, and why does the middle look so blurry?

The **lines are artifacts (boundary effect) from the warping method**. When the image is warped with interpolation, edge pixels containing part image and part background combine these values. We would have bright lines if we'd chosen `cval=2` in the `warp` calls (try it!), but regardless of choice there will always be discontinuities.

...Unless you use `order=0` in `warp`, which is nearest neighbor. Then edges are perfect (try it!). But who wants to be limited to an inferior interpolation method?

Even then, it's blurry! Is there a better way?

---

## 7 5. Stitching images along a minimum-cost path

Let's step back a moment and consider: Is it even reasonable to blend pixels?

Take a look at a difference image, which is just one image subtracted from the other.

```
[29]: fig, ax = plt.subplots(figsize=(15,12))

# Generate difference image and inspect it
difference_image = pano0_warped - pano1_warped
ax.imshow(difference_image, cmap='gray')

ax.axis('off');
```



The surrounding flat gray is zero. *A perfect overlap would show no structure!*

Instead, the overlap region matches fairly well in the middle... but off to the sides where things start to look a little embossed, a simple average blurs the result. This caused the blurring in the previous, method (look again). *Unfortunately, this is almost always the case for panoramas!*

How can we fix this?

Let's attempt to find a vertical path through this difference image which stays as close to zero as possible. If we use that to build a mask, defining a transition between images, the result should appear *seamless*.

## 8 Seamless image stitching with Minimum-Cost Paths and `skimage.graph`

Among other things, `skimage.graph` allows you to \* start at any point on an array \* find the path to any other point in the array \* the path found *minimizes* the sum of values on the path.

The array is called a *cost array*, while the path found is a *minimum-cost path* or **MCP**.

To accomplish this we need

- Starting and ending points for the path
- A cost array (a modified difference image)

This method is so powerful that, with a carefully constructed cost array, the seed points are essentially irrelevant. It just works!

### 8.0.1 Define seed points

```
[30]: rmax = output_shape[0] - 1
      cmax = output_shape[1] - 1

      # Start anywhere along the top and bottom, left of center.
      mask_pts01 = [[0,      cmax // 3],
                   [rmax, cmax // 3]]

      # Start anywhere along the top and bottom, right of center.
      mask_pts12 = [[0,      2*cmax // 3],
                   [rmax, 2*cmax // 3]]
```

### 8.0.2 Construct cost array

This utility function exists to give a “cost break” for paths from the edge to the overlap region.

We will visually explore the results shortly. Examine the code later - for now, just use it.

```
[31]: from skimage.morphology import flood_fill

def generate_costs(diff_image, mask, vertical=True, gradient_cutoff=2.):
    """
    Ensures equal-cost paths from edges to region of interest.

    Parameters
    -----
    diff_image : ndarray of floats
        Difference of two overlapping images.
    mask : ndarray of bools
        Mask representing the region of interest in ``diff_image``.
    vertical : bool
        Control operation orientation.
    gradient_cutoff : float
```



*Controls how far out of parallel lines can be to edges before correction is terminated. The default (2.) is good for most cases.*

*Returns*

-----

*costs\_arr : ndarray of floats*

*Adjusted costs array, ready for use.*

"""

**if** vertical **is not** True:

**return** tweak\_costs(diff\_image.T, mask.T, vertical=vertical,  
                        gradient\_cutoff=gradient\_cutoff).T

*# Start with a high-cost array of 1's*

costs\_arr = np.ones\_like(diff\_image)

*# Obtain extent of overlap*

row, col = mask.nonzero()

cmin = col.min()

cmax = col.max()

shape = mask.shape

*# Label discrete regions*

labels = mask.copy().astype(np.uint8)

cslice = slice(cmin, cmax + 1)

submask = np.ascontiguousarray(labels[:, cslice])

submask = flood\_fill(submask, (0, 0), 2)

submask = flood\_fill(submask, (shape[0]-1, 0), 3)

labels[:, cslice] = submask

*# Find distance from edge to region*

upper = (labels == 2).sum(axis=0).astype(np.float64)

lower = (labels == 3).sum(axis=0).astype(np.float64)

*# Reject areas of high change*

ugood = np.abs(np.gradient(upper[cslice])) < gradient\_cutoff

lgood = np.abs(np.gradient(lower[cslice])) < gradient\_cutoff

*# Give areas slightly farther from edge a cost break*

costs\_upper = np.ones\_like(upper)

costs\_lower = np.ones\_like(lower)

costs\_upper[cslice][ugood] = upper[cslice].min() / np.

↪maximum(upper[cslice][ugood], 1)

costs\_lower[cslice][lgood] = lower[cslice].min() / np.

↪maximum(lower[cslice][lgood], 1)

*# Expand from 1d back to 2d*

vdist = mask.shape[0]

```

costs_upper = costs_upper[np.newaxis, :].repeat(vdist, axis=0)
costs_lower = costs_lower[np.newaxis, :].repeat(vdist, axis=0)

# Place these in output array
costs_arr[:, cslice] = costs_upper[:, cslice] * (labels[:, cslice] == 2)
costs_arr[:, cslice] += costs_lower[:, cslice] * (labels[:, cslice] == 3)

# Finally, place the difference image
costs_arr[mask] = diff_image[mask]

return costs_arr

```

Use this function to generate the cost array.

```

[32]: # Start with the absolute value of the difference image.
      # np.abs is necessary because we don't want negative costs!
      costs01 = generate_costs(np.abs(pano0_warped - pano1_warped),
                              pano0_mask & pano1_mask)

```

Allow the path to “slide” along top and bottom edges to the optimal horizontal position by setting top and bottom edges to zero cost.

```

[33]: costs01[0, :] = 0
      costs01[-1, :] = 0

```

Our cost array now looks like this

```

[34]: fig, ax = plt.subplots(figsize=(15, 12))

      ax.imshow(costs01, cmap='gray', interpolation='none')

      ax.axis('off');

```



The tweak we made with `generate_costs` is subtle but important. Can you see it?

### 8.0.3 Find the minimum-cost path (MCP)

Use `skimage.graph.route_through_array` to find an optimal path through the cost array

```
[35]: from skimage.graph import route_through_array

# Arguments are:
#   cost array
#   start pt
#   end pt
#   can it traverse diagonally
pts, _ = route_through_array(costs01, mask_pts01[0], mask_pts01[1],
                             ↪fully_connected=True)

# Convert list of lists to 2d coordinate array for easier indexing
pts = np.array(pts)
```

Did it work?

```
[36]: fig, ax = plt.subplots(figsize=(12, 12))

# Plot the difference image
ax.imshow(pano0_warped - pano1_warped, cmap='gray')
```

```
# Overlay the minimum-cost path
ax.plot(pts[:, 1], pts[:, 0])

plt.tight_layout()
ax.axis('off');
```



That looks like a great seam to stitch these images together - the path looks very close to zero.

#### 8.0.4 Irregularities

Due to the random element in the RANSAC transform estimation, everyone will have a slightly different path. **Your path will look different from mine, and different from your neighbor's.** That's expected! *The awesome thing about MCP is that everyone just calculated the best possible path to stitch together their unique transforms!*

#### 8.0.5 Filling the mask

Turn that path into a mask, which will be 1 where we want the left image to show through and zero elsewhere. We need to fill the left side of the mask with ones over to our path.

**Note:** This is the inverse of NumPy masked array conventions (`numpy.ma`), which specify a negative mask (`mask == bad/missing`) rather than a positive mask as used here (`mask == good/selected`).

Place the path into a new, empty array.

```
[37]: # Start with an array of zeros and place the path
mask0 = np.zeros_like(pano0_warped, dtype=np.uint8)
mask0[pts[:, 0], pts[:, 1]] = 1
```

Ensure the path appears as expected

```
[38]: fig, ax = plt.subplots(figsize=(11, 11))

# View the path in black and white
ax.imshow(mask0, cmap='gray')

ax.axis('off');
```

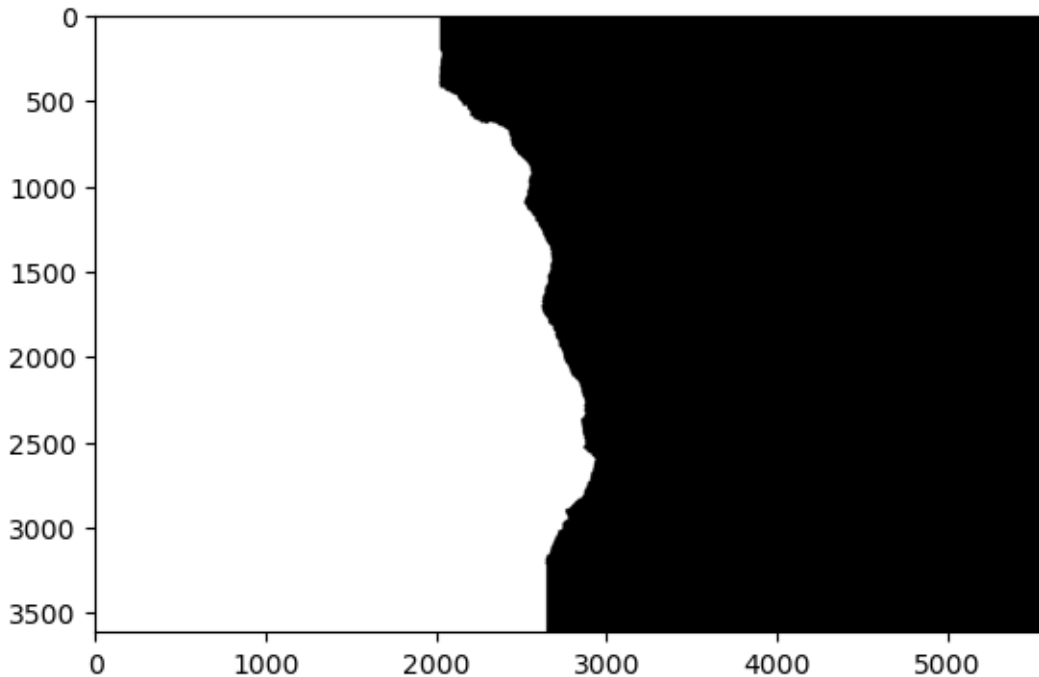


Label the various contiguous regions in the image using `skimage.measure.label`

```
[39]: from skimage.morphology import flood_fill

# Labeling starts with one at point (0, 0)
mask0 = flood_fill(mask0, (0, 0), 1, connectivity=1)

# The result
plt.imshow(mask0, cmap='gray');
```



Looks great!

### 8.0.6 Rinse and repeat

Apply the same principles to images 1 and 2: first, build the cost array

```
[40]: # Start with the absolute value of the difference image.
# np.abs necessary because we don't want negative costs!
costs12 = generate_costs(np.abs(pano1_warped - pano2_warped),
                        pano1_mask & pano2_mask)

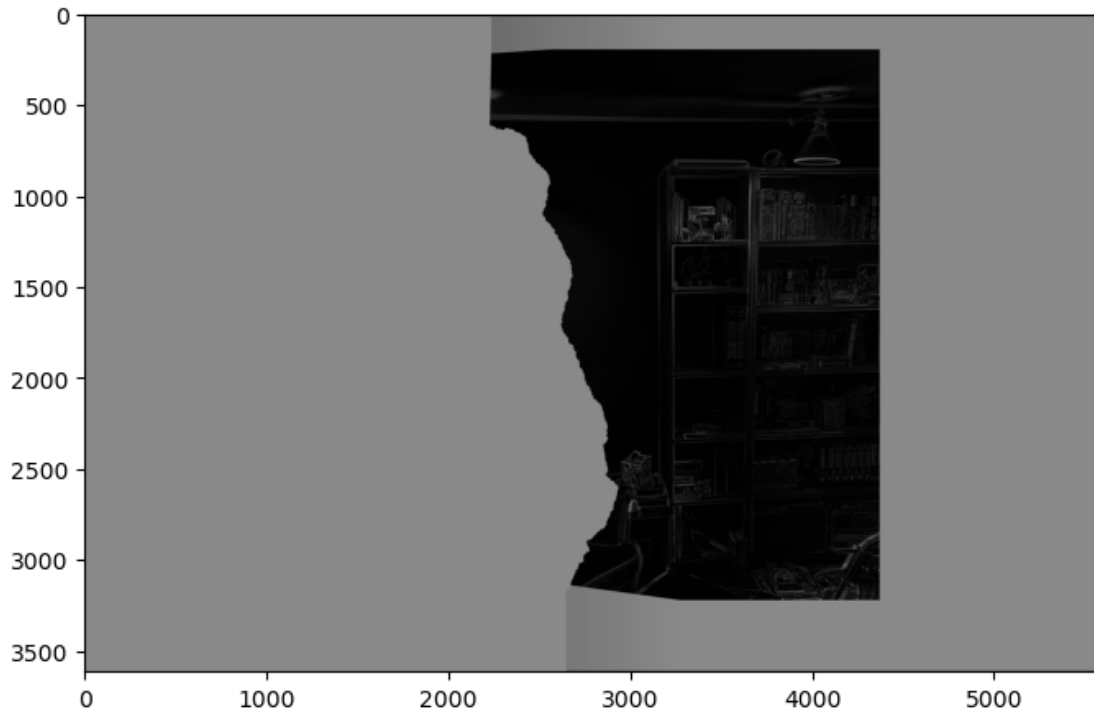
# Allow the path to "slide" along top and bottom edges to the optimal
# horizontal position by setting top and bottom edges to zero cost
costs12[0, :] = 0
costs12[-1, :] = 0
```

Add an additional constraint this time, to prevent this path crossing the prior one!

```
[41]: costs12[mask0 > 0] = 1
```

Check the result

```
[42]: fig, ax = plt.subplots(figsize=(8, 8))
ax.imshow(costs12, cmap='gray');
```



Your results may look slightly different.

Compute the minimal cost path

```
[43]: # Arguments are:
#      cost array
#      start pt
#      end pt
#      can it traverse diagonally
pts, _ = route_through_array(costs12, mask_pts12[0], mask_pts12[1],
    ↪fully_connected=True)

# Convert list of lists to 2d coordinate array for easier indexing
pts = np.array(pts)
```

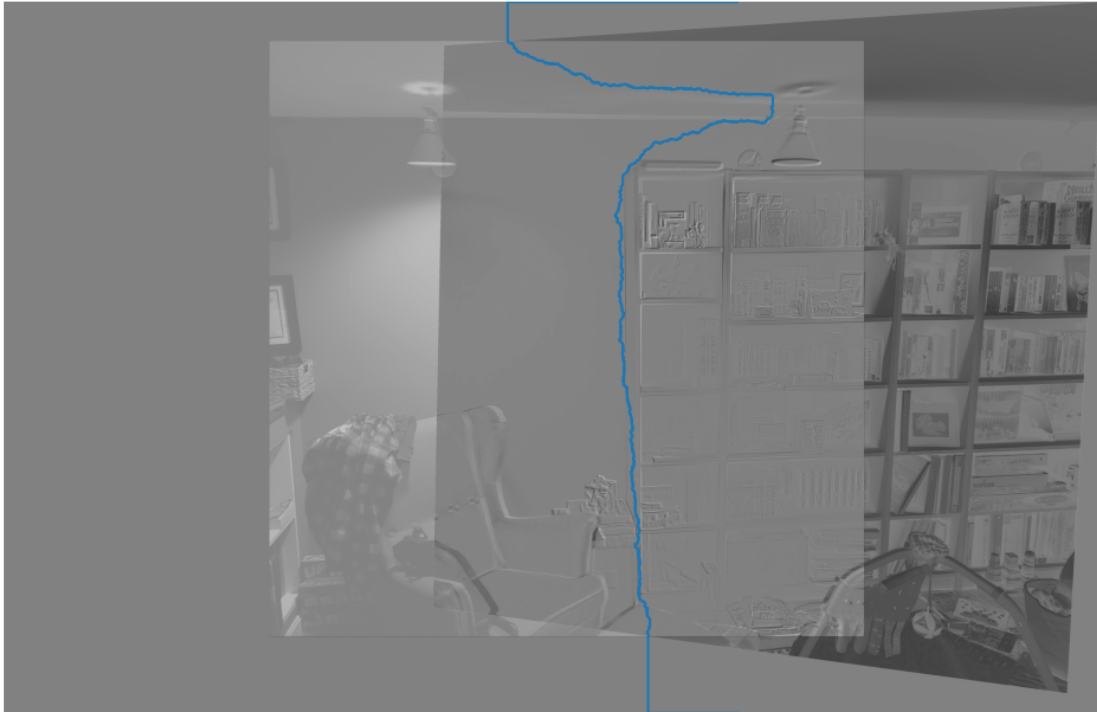
Verify a reasonable result

```
[44]: fig, ax = plt.subplots(figsize=(12, 12))

# Plot the difference image
ax.imshow(pano1_warped - pano2_warped, cmap='gray')

# Overlay the minimum-cost path
ax.plot(pts[:, 1], pts[:, 0]);
```

```
ax.axis('off');
```



Initialize the mask by placing the path in a new array

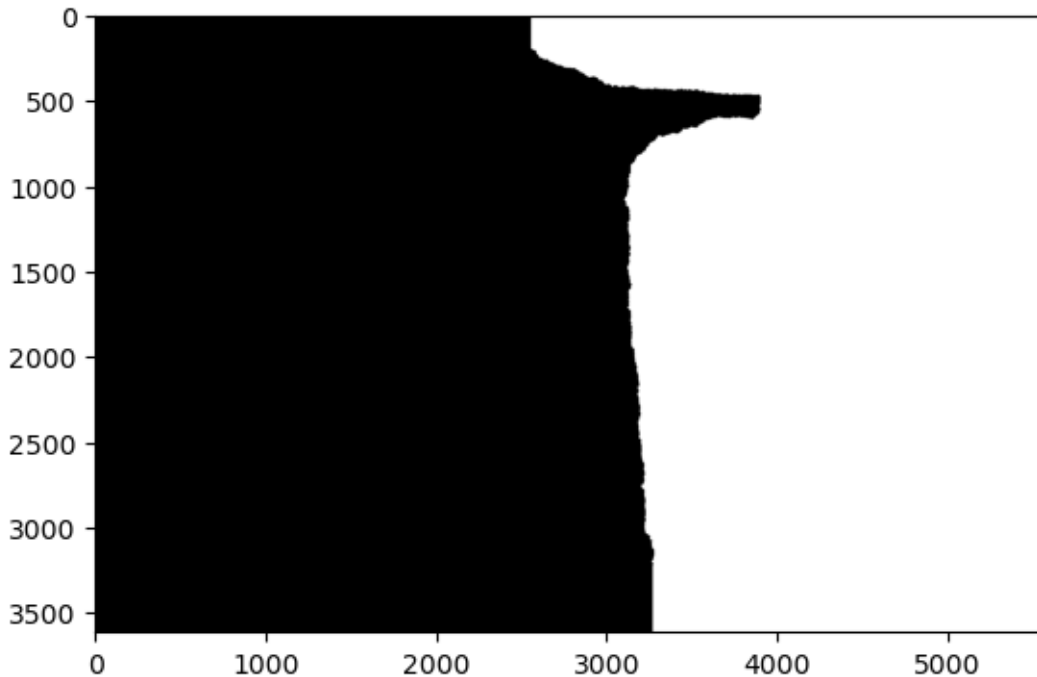
```
[45]: mask2 = np.zeros_like(pano0_warped, dtype=np.uint8)
      mask2[pts[:, 0], pts[:, 1]] = 1
```

Fill the right side this time, again using `skimage.measure.label` - the label of interest is 2

```
[46]: mask2 = flood_fill(mask2, (rmax, cmax), 1, connectivity=1)

      # The result
      plt.imshow(mask2, cmap='gray');
```





### 8.0.7 Final mask

The last mask for the middle image is one of exclusion - it will be displayed everywhere `mask0` and `mask2` are not.

```
[48]: mask1 = ~(mask0.astype(bool) | mask2.astype(bool))
```

Define a convenience function to place masks in alpha channels

```
[49]: def add_alpha(img, mask=None):
    """
    Adds a masked alpha channel to an image.

    Parameters
    -----
    img : (M, N[, 3]) ndarray
        Image data, should be rank-2 or rank-3 with RGB channels
    mask : (M, N[, 3]) ndarray, optional
        Mask to be applied. If None, the alpha channel is added
        with full opacity assumed (1) at all locations.
    """
    from skimage.color import gray2rgb
    if mask is None:
        mask = np.ones_like(img)
```

```

if img.ndim == 2:
    img = gray2rgb(img)

return np.dstack((img, mask))

```

Obtain final, alpha blended individual images and inspect them

```

[50]: pano0_final = add_alpha(pano0_warped, mask0)
      pano1_final = add_alpha(pano1_warped, mask1)
      pano2_final = add_alpha(pano2_warped, mask2)

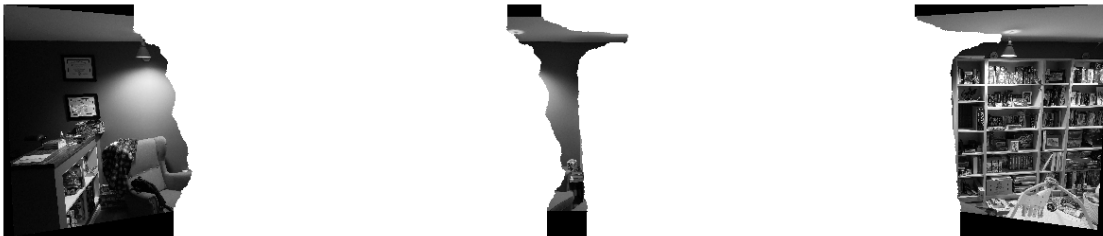
      compare(pano0_final, pano1_final, pano2_final, figsize=(12, 12))

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



What we have here is the world's most complicated and precisely-fitting jigsaw puzzle...

Plot all three together and view the results!

```

[51]: fig, ax = plt.subplots(figsize=(12, 12))

      # This is a perfect combination, but matplotlib's interpolation
      # makes it appear to have gaps. So we turn it off.
      ax.imshow(pano0_final, interpolation='none')
      ax.imshow(pano1_final, interpolation='none')
      ax.imshow(pano2_final, interpolation='none')

      fig.tight_layout()
      ax.axis('off');

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



Fantastic! Without the black borders, you'd never know this was composed of separate images!

---

## 9 Bonus round: now, in color!

We converted to grayscale for ORB feature detection, back in the initial **preprocessing** steps. Since we stored our transforms and masks, adding color is straightforward!

Transform the colored images

```
[ ]: # Identical transforms as before, except
#     * Operating on original color images
#     * filling with cval=0 as we know the masks
pano0_color = warp(pano_imgs[0], (model_robust01 + offset1).inverse, order=3,
                   output_shape=output_shape, cval=0)

pano1_color = warp(pano_imgs[1], offset1.inverse, order=3,
                   output_shape=output_shape, cval=0)

pano2_color = warp(pano_imgs[2], (model_robust12 + offset1).inverse, order=3,
                   output_shape=output_shape, cval=0)
```

Apply the custom alpha channel masks

```
[ ]: pano0_final = add_alpha(pano0_color, mask0)
     pano1_final = add_alpha(pano1_color, mask1)
     pano2_final = add_alpha(pano2_color, mask2)
```

View the result!

```
[ ]: fig, ax = plt.subplots(figsize=(12, 12))

     # Turn off matplotlib's interpolation
     ax.imshow(pano0_final, interpolation='none')
     ax.imshow(pano1_final, interpolation='none')
     ax.imshow(pano2_final, interpolation='none')

     fig.tight_layout()
     ax.axis('off');
```

Save the combined, color panorama locally as './pano-advanced-output.png'

```
[ ]: from skimage.color import gray2rgb

     # Start with empty image
     pano_combined = np.zeros_like(pano0_color)

     # Place the masked portion of each image into the array
     # masks are 2d, they need to be (M, N, 3) to match the color images
     pano_combined += pano0_color * gray2rgb(mask0)
     pano_combined += pano1_color * gray2rgb(mask1)
     pano_combined += pano2_color * gray2rgb(mask2)

     # Save the output - precision loss warning is expected
     # moving from floating point -> uint8
     io.imsave('./pano-advanced-output.png', pano_combined)
```

---

#Once more, from the top

I hear what you're saying. "Those were too easy! The panoramas had too much overlap! Does this still work in the real world?"

Go back to the top. Under "Load Data" replace the string 'data/JDW\_03\*' with 'data/JDW\_9\*', and re-run all of the cells in order.

---

```
[ ]: %reload_ext load_style
     %load_style ../../themes/tutorial.css
```

[ ]: